

Symbolic Execution Engine for BTOR Systems

Kevin Chen

March 20, 2026

1 Introduction

Verifying that hardware and software systems behave correctly under all possible inputs is a fundamental challenge in computer science. Traditional testing approaches require engineers to enumerate concrete inputs by hand, which makes it difficult to achieve complete coverage of all possible behaviors. Formal verification methods address this limitation by reasoning about all inputs simultaneously, but they often require significant manual effort to apply.

This paper presents a symbolic execution engine for transition systems described in the BTOR2 format, a word-level model checking language widely used for hardware verification. Rather than running a system on one concrete input at a time, our engine keeps inputs symbolic and explores all feasible execution paths in parallel, using an SMT solver to determine which branches are reachable and to detect violations of safety properties. When a bad state is reachable, the engine reports both the symbolic expression that was violated and the path condition that witnesses the violation.

1.1 Symbolic Execution

Symbolic execution is a way of executing a program abstractly, such that on abstract execution covers multiple possible inputs of the program that share a particular execution path through the code [1]. Each input of symbolically executed code is represented as a symbol rather than assigning it a concrete value. When the value is referenced such as for an if statement, the execution path branches, where one path the symbolic value is evaluated as true while the other path the symbolic value is evaluated as false.

1.1.1 Symbolic Execution in Industry

One of the key uses for symbolic execution is in testing. Most unit tests, require the programmer to hardcode in values to test, resulting in various unit tests to cover all branches the code can tranverse. On the other hand, a symbolic execution would execute simultaneously for all possible inputs. Algorithm 1 gives the following example.

Algorithm 1 Algorithm 1

Require: $x, y \in \mathbb{Z}$

```
1:  $a \leftarrow x + y$ 
2: if  $a \bmod 2 = 0$  then
3:   return error “ $a$  is even”
4: else
5:   return  $a$ 
6: end if
```

With a glance, we can see that using any combination of numbers such that their sum results in an even number will result in an error. There is no way to cover all cases of failure using unit tests, as there are an infinite number of way which we can stumble onto this error. Instead of testing with unit tests such as $x = 1, y = 1$, symbolic execution represents the inputs as symbols $x = x_0, y = y_0$. Consider a symbolic execution of the algorithm above; Figure 1 shows how we start with representing each input symbolically.

line	Path Condition	Variable Assignments
0	true	$x \rightarrow x_0, y \rightarrow y_0$
1	true	$x \rightarrow x_0, y \rightarrow y_0, a \rightarrow x_0 + y_0$
2	$(x_0 + y_0) \bmod 2 = 0$	$x \rightarrow x_0, y \rightarrow y_0, a \rightarrow x_0 + y_0$
2	$\neg((x_0 + y_0) \bmod 2 = 0)$	$x \rightarrow x_0, y \rightarrow y_0, a \rightarrow x_0 + y_0$
3	$(x_0 + y_0) \bmod 2 = 0$	Error
5	$\neg((x_0 + y_0) \bmod 2 = 0)$	Return $x_0 + y_0$

Figure 1: Symbolic execution trace showing path conditions and variable translations

In this example, we see that the inputs values start symbolic as symbols which has no concrete value. Following the execution, we translate every instance of x with x_0 with the same case for y . Where it gets interesting is when the the execution has a branching path. In the if statement, because a is symbolic, the executor cannot evaluate the condition concretely. Instead, execution forks into two paths, each assuming a different outcome of the condition and adding the corresponding constraint to the path condition. Figure 2 shows the different execution paths the execution takes via a simple tree.

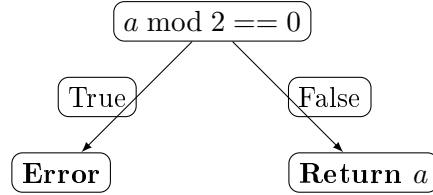


Figure 2: Execution tree showing the branching paths in symbolic execution

In each paths, we accumulate the different path conditions until we either reach the end of function or a failure. Our symbolic execution analysis shows us that there are two possible families of results: one which results in an error and one in a valid return value. Tracing its path constraint reviews to us that the family of inputs satisfy the constraint $a \bmod 2 == 0$ results in an error. Now imagine a similar example, but with a twist. Prior to running this program with symbolic execution, one can add constraints to the path. Algorithm 2 gives an example where we start with a given constraint that $x_0 + y_0 = 1$.

Algorithm 2 Algorithm 1 called with a precondition

Require: $x, y \in \mathbb{Z}$

- 1: **if** $x + y = 1$ **then**
 - 2: ALGORITHM1(x, y)
 - 3: **end if**
-

With the constraint, intuitively we can see that the possibility to return an error is impossible.

Symbolic execution reflects this by maintaining a path of path constraints and pruning out infeasible outcomes (Figure 3).

line	Path Condition	Translation
0	$x_0 + y_0 = 1$	$x \rightarrow x_0, y \rightarrow y_0$
1	$x_0 + y_0 = 1$	$x \rightarrow x_0, y \rightarrow y_0, a \rightarrow x_0 + y_0$
2	$\neg((x_0 + y_0) \bmod 2 = 0)$	$x \rightarrow x_0, y \rightarrow y_0, a \rightarrow x_0 + y_0$
2	$x_0 + y_0 = 1 \wedge (x_0 + y_0) \bmod 2 = 0$	$x \rightarrow x_0, y \rightarrow y_0, a \rightarrow x_0 + y_0$
3	False	Error
5	$\neg((x_0 + y_0) \bmod 2 = 0)$	Return $x_0 + y_0$

Figure 3: Modified version of execution with existing constraints

With the constraint, the path condition which leads to the error is contradictory, which shows that the path is impossible and no paths lead to errors. In industry, programmers, can create tests with given preconditions (constraints) and run until the program terminates. Instead of throwing an error, we can use assertion statements as post-conditions. If it terminates without an error, the program is robust across all set of inputs. Otherwise, a symbolically executed program will return a concrete set of conditions which will give an error or violate a post-condtioon.

1.1.2 While loops

A while loop can be viewed as repeated conditional branching. At each iteration, the loop condition is evaluated, and execution either continues into the loop body or exits the loop. In this sense, a while loop behaves like an if statement that is evaluated repeatedly, where each evaluation produces a new branch in the execution path. Because symbolic execution already handles branching through constructs such as if statements, loops can be handled in a variety of ways. In imperative symbolic execution engines, loops are typically handled directly by repeatedly exploring these branches. In declarative or functional settings, another common approach is to represent the loop by unrolling it into a recursive function. This removes the explicit loop construct while preserving the same branching behavior. For illustration, see Algorithm 3.

Algorithm 3 Count Number of 1's in a Bit Vector

Require: v is a bit vector of length n

```

1:  $count \leftarrow 0$ 
2:  $i \leftarrow 0$ 
3: while  $i < n$  do
4:   if  $v[i] == 1$  then
5:      $count \leftarrow count + 1$ 
6:   end if
7:    $i \leftarrow i + 1$ 
8: end while
9: return  $count$ 
```

Algorithm 4 rewrites it as a recursive function. In practice, splitting branches continuous may lead to state explosion, where thousands of branches consume memory within iterations. To combat this, programmers either set a recursion limit, terminating the branch which gets stuck, or find a different way to represent the while loop, using the loop invariant. Figure 4 shows a tree of the branching paths created from a symbolic execution of Algorithm 4.

Algorithm 4 Recursively Count Number of 1's in a Bit Vector

Require: v is a bit vector of length n

```
1: function COUNT_ONES( $v, i, n$ )
2:   if  $i == n$  then
3:     return 0
4:   else if  $v[i] == 1$  then
5:     return  $1 + \text{COUNT\_ONES}(v, i + 1, n)$ 
6:   else
7:     return  $\text{COUNT\_ONES}(v, i + 1, n)$ 
8:   end if
9: end function
10: return  $\text{COUNT\_ONES}(v, 0, n)$ 
```

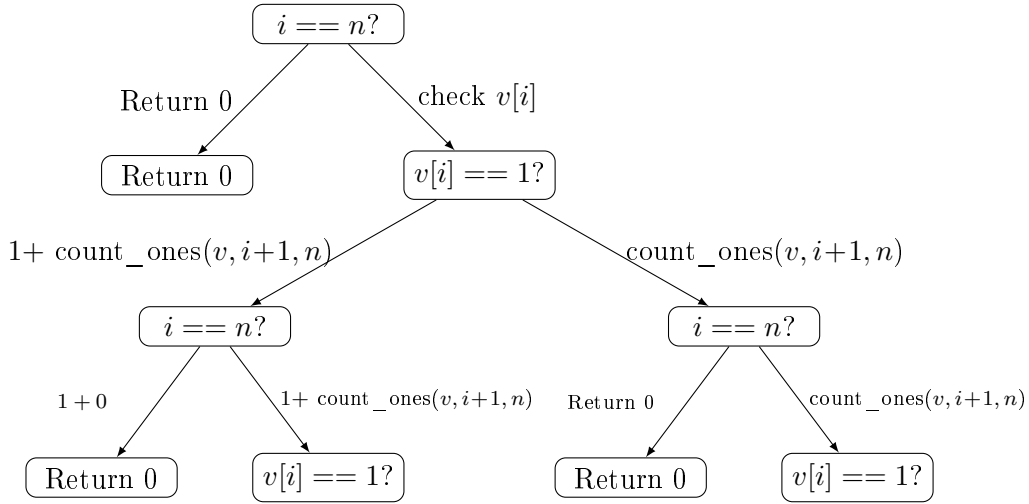


Figure 4: Symbolic execution tree for the recursive `count_ones` algorithm showing branching at each condition

1.2 BTOR2

BTOR2 is a word-level model checking format for capturing models of hardware and potentially software in a bit-precise manner. The exact architecture in which it is defined can be found at [3].

For the purposes of this project, we parse the language into a *transition system*, formalized as a tuple $(\mathcal{S}, \mathcal{I}, \text{init}, \text{next}, \mathcal{C}, \mathcal{B})$:

- $\mathcal{S}$: the set of state variables;
- $\mathcal{I}$: the set of free (input) variables;
- *init*: for each $s \in \mathcal{S}$, the initial value $\text{init}(s)$;
- *next*: for each state variable $s \in \mathcal{S}$, the next-state function $\text{next}(s, \mathcal{I})$ giving the value of s after one step as a function of the current state and chosen inputs;
- $\mathcal{C}$: the set of constraints that every chosen input must satisfy at each step; and

- $\mathcal{B}$: the bad-state predicate; the system is unsafe if some execution reaches a state satisfying $\mathcal{B}$.

These components map directly to BTOR2: `state` (with `init` and `next`), `input`, `constraint`, and `bad`.

The BTOR interpreter runs the transition system step by step: it initializes each state variable using *init*, then at each step chooses concrete values for the inputs in $\mathcal{I}$ that satisfy the constraints in $\mathcal{C}$, evaluates each next-state function in *next*, and updates the state accordingly. After each step, it checks whether the resulting state satisfies the bad-state predicate $\mathcal{B}$.

Consider a system with one state variable s (initialized to 0, $next(s) = s + inp$), one free input inp constrained by $inp \geq 1$, and bad state $\mathcal{B} \equiv s \geq 3$. Table 5 shows a concrete two-step run that reaches a bad state.

Step	State (s)	Bad? ($s \geq 3$)	Input (inp)	Next State (s')
Init	0	false	1	1
1	1	false	2	3
2	3	true	—	—

Figure 5: Two-step BTOR2 execution trace. State s is initialized to 0; each step applies a chosen input (constrained to ≥ 1) and updates s via *next*. The bad predicate $s \geq 3$ is first satisfied at step 2, witnessing the safety violation.

1.3 SMT Solver

One of the key challenges of implementing symbolic execution is being able to distinguish whether or not a path is feasible or not. How should a symbolic executor tell whether or not a path is feasible such as in Figure 3 row 4 or infeasible, like in Figure 3 row 3.

As always in computer science, one of the best ways to model something is to make it someone else’s problem: take an existing solution to a hard problem and leverage it to solve your own. Here, we leverage a SAT solver to determine path feasibility. The remaining challenge is translating each branching condition into a series of boolean equations. This is the role of an SMT (Satisfiability Modulo Theories) solver, which lifts raw SAT solving to richer data types (e.g., integers, bit-vectors).

One possible way to reduce bit-vector constraints to SAT: model fixed-width integers as bit-vectors (e.g., a 32-bit integer as 32 boolean variables) and encode comparisons as propositional formulas. As a concrete example of such a reduction, consider a 4-bit unsigned integer $x = (x_3, x_2, x_1, x_0)$, where x_3 is the most significant bit. One way to encode the condition $x > 4$ is to observe that $4 = (0, 1, 0, 0)_2$ and reason case-by-case over the leading bits: if $x_3 = 1$ then $x \geq 8 > 4$; if $x_3 = 0$ and $x_2 = 1$ then $x \in \{4, \dots, 7\}$, and $x > 4$ requires at least one of x_1, x_0 to be 1; otherwise $x \leq 3$. This yields the propositional formula

$$x_3 \vee (x_2 \wedge (x_1 \vee x_0))$$

which can be passed to a SAT solver. Thus, bit-vector SMT can be lowered to SAT in this way; actual SMT solvers may use different encodings or avoid full bit-blasting.

1.3.1 Bounded Model Checking

Rather than implementing a symbolic execution engine, a simpler way of checking program correctness is with Bounded Model Checking(BMC). Here, this verification system translates the program into a system of SMT queries and checks if there exists an example which results in an error.

Using the transition system style from Table 5, BMC keeps the inputs symbolic and asks whether there exists an assignment within the chosen bound that reaches a bad state. For example, let the initial state be 1, let $next(s) = s + inp$, and let the bad predicate be $s \geq 4$. Unrolling the system for one step yields Table 6. The check fails within one step because choosing $i_0 = 3$ gives the next state $1 + i_0 = 4$, which satisfies the bad predicate.

step	state	input	next state	bad
Init	1	i_0	$i_0 + 1$	—
1	$1 + i_0$	i_1	$i_1 + i_0 + 1$	yes

Figure 6: One-step bounded model checking trace for the transition system induced by Table 5. The inputs remain symbolic, and the checker asks whether some assignment, such as $i_0 = 3$, makes the bad predicate true within the bound.

This technique is called BMC as it only checks for a finite number of steps. Suppose that the input was constrained such that $0 < i < 2$ and that the bad state required the state be > 10 . With a maximum of 9 steps, BMC would stop after 9 step concluding that there exists no error.

2 Implementation

The high-level goal of our implementation is to perform symbolic execution on BTOR2 transition systems. In this setting, the configuration of the system is the valuation of all its state variables: each state variable holds a value (concrete or symbolic), and two configurations are equivalent if they assign the same value to every state variable. To compute the next step, each configuration is transformed according to the transition function: the next-state expression for each state variable is evaluated, possibly using symbolic inputs, and the result becomes that variable’s new value.

Because inputs can be nondeterministic and next-state expressions may contain **ite** (conditional) branches, a single configuration can lead to multiple successor configurations. We therefore maintain a set of paths, where each path is a pair of (i) a configuration, the current symbolic values of all state variables, and (ii) a path condition: the conjunction of assumptions made to reach that configuration. Given a system and a step bound, we initialize the set of paths, repeatedly apply the transition function to each path’s configuration to obtain new paths, and check whether any path reaches a configuration satisfying the bad-state predicate.

2.1 Step-by-Step Example

First we illustrate how the symbolic executor runs with an example BTOR2 model. First, the transition system is given by Algorithm 5.

Step 0 The executor begins with the initial configuration. Step 0 starts with one path and no assumptions (path condition *true*). We set up the state variables: the BTOR2 model gives **counter** no explicit initial value, so we assign it a fresh timestamped symbol **counter_t0** (16-bit). **_resetCount** (1-bit) is initialized to 0 as specified. At each step, the input **reset** is chosen nondeterministically and represented by a timestamped symbol (e.g., **reset_t0** at step 0, **reset_t1** at step 1). The system constraint $implies(\neg(_resetCount \geq 1), reset)$ has antecedent true (since $_resetCount = 0$), so **reset** must hold. Thus the path condition includes **reset**. The table below shows the single path: **counter** = **counter_t0**, **_resetCount** = 0.

Path	Constraints	counter (16-bit)	_resetCount (1-bit)
1	reset	counter_t0	0

Algorithm 5 Quiz1 transition system

- 1: **Input:** `reset` (1-bit)
 - 2: **State:** `counter` (16-bit), `_resetCount` (1-bit)
 - 3: **Init:** `_resetCount` $\leftarrow$ 0
 - 4: **Constraint:** $\text{implies}(\neg(\text{_resetCount} \geq 1), \text{reset})$
 - 5: **Bad:** $\neg \text{reset} \wedge (\text{counter} > 1)$
 - 6: **Next:**
 - 7: `counter` $\leftarrow$ `ite(reset, 0, counter + 1)`
 - 8: `_resetCount` $\leftarrow$ `ite(_resetCount < 1, 1, _resetCount)`
-

Step 1 We take each path produced at step 0 and apply the transition function to it. Step 0 had one path, so we have one path to process. We introduce `reset_t0` for the input at this step. The constraint forced `reset` to hold at step 0, so we are in the “reset” branch of the `ite`: `counter` becomes 0 and `_resetCount` becomes 1. After the transition, `_resetCount` = 1, so $\neg(\text{_resetCount} \geq 1)$ is false and the system constraint is vacuously satisfied (simplifies to *true*). The path condition remains `reset` (or *true*). We still have one path: `counter` = 0, `_resetCount` = 1.

Path	Constraints	counter (16-bit)	_resetCount (1-bit)
1	<i>true</i>	0	1

Step 2 We take each path from step 1 and apply the transition. Step 1 had one path, so we start with one path. We introduce `reset_t1` for the input at this step. The next-state expression for `counter` is `ite(reset, 0, counter + 1)`. We cannot evaluate this concretely because `reset` = `reset_t1` is symbolic. We create two successor paths, one assuming `reset_t1` = *true*, one assuming `reset_t1` = *false*. **Path 1** (reset held): `reset_t1` = *true*, so `counter` stays 0 and `_resetCount` stays 1. Path condition: `reset_t1`. **Path 2** (reset false): `reset_t1` = *false*, so `counter` becomes 1 and `_resetCount` stays 1. Path condition: $\neg \text{reset_t1}$. We now have two paths, each inheriting from the single path at step 1 but branching on the symbolic input.

Path	Constraints	counter (16-bit)	_resetCount (1-bit)
1	<code>reset_t1</code>	0	1
2	<code>not(reset_t1)</code>	1	1

Step 3 We take each of the two paths from step 2 and apply the transition. For each path, we introduce `reset_t2` for the input at this step. Each path branches on `reset_t2` (true or false), yielding four paths with no merging. **Path 1** continues from step 2 path 1 with `reset_t2` = *true*: `counter` stays 0. Path condition: `reset_t1` $\wedge$ `reset_t2`. **Path 2** continues from step 2 path 1 with `reset_t2` = *false*: `counter` was 0, so it becomes 1. Path condition: `reset_t1` $\wedge$ $\neg \text{reset_t2}$. **Path 3** continues from step 2 path 2 with `reset_t2` = *true*: `counter` was 1, so it becomes 0. Path condition: $\neg \text{reset_t1}$ $\wedge$ `reset_t2`. **Path 4** continues from step 2 path 2 with `reset_t2` = *false*: `counter` was 1, so it becomes 2. Path condition: $\neg \text{reset_t1}$ $\wedge$ $\neg \text{reset_t2}$. The bad-state predicate is $\neg \text{reset} \wedge (\text{counter} > 1)$. On path 4, `reset_t2` = *false* (so $\neg \text{reset}$ holds) and `counter` = 2 > 1, so the bad state is satisfied. The executor reports that the system is unsafe: a concrete trace (e.g., `reset_t0` = *true*, `reset_t1` = *false*, `reset_t2` = *false*) reaches the bad state at step 3.

Path	Constraints	counter (16-bit)	_resetCount (1-bit)
1	<code>reset_t1, reset_t2</code>	0	1
2	<code>reset_t1, not(reset_t2)</code>	1	1
3	<code>not(reset_t1), reset_t2</code>	0	1
4	<code>not(reset_t1), not(reset_t2)</code>	2	1

2.2 Initialization

Initialization sets up the data structures that the symbolic executor will maintain throughout execution. Before describing the procedure itself, we first examine the two core types it operates on: the `SymbolicExecutor` struct, which holds all persistent state, and the `ExecutionPath` struct, which represents a single feasible branch.

2.2.1 SymbolicExecutor Fields

The `SymbolicExecutor` struct is the top-level object. It is constructed from a parsed BTOR2 transition system and owns all data that persists across steps:

```
pub struct SymbolicExecutor {
    ts: TransitionSystem,
    current_step: usize,
    paths: Vec<ExecutionPath>,
    state_types: HashMap<ExprRef, Type>,
    input_types: HashMap<ExprRef, Type>,
    all_symbols: HashSet<ExprRef>,
    constraints: Vec<ExprRef>,
    bad_state: Option<(ExprRef, ExecutionPath)>,
}
```

The fields serve the following roles:

- **ts** — the parsed BTOR2 model, including the state variables, inputs, next-state expressions, constraints, and bad-state predicates.
- **current_step** — the number of transitions that have been applied so far, used to generate unique timestamped names for new symbols.
- **paths** — the set of active symbolic execution paths. Each path represents one feasible branch of execution reached so far.
- **state_types / input_types** — maps from expression references to their BTOR2 types (bit-vector width or array type), consulted whenever a fresh symbol must be allocated with the correct type.
- **all_symbols** — the universe of all symbolic variables that have been introduced. Every variable declared here is forwarded to the SMT solver before checking satisfiability.
- **constraints** — the system-level constraints copied from the transition system. These are asserted in every SMT query so that only inputs satisfying the BTOR2 `constraint` declarations are considered.
- **bad_state** — set to `Some` the first time a bad-state predicate is found satisfiable on any path, pairing the violating expression with the witness path. Subsequent calls to `step` return immediately with `StepResult::BadStateReached` without modifying any other field.

Each active branch is stored as an `ExecutionPath`:


```

pub struct ExecutionPath {
  pub state_translation: HashMap<ExprRef, ExprRef>,
  pub path_conditions: Vec<ExprRef>,
  pub input_translation: HashMap<ExprRef, ExprRef>,
  pub new_state_cache: HashMap<ExprRef, ExprRef>,
}

```

`state_translation` maps each state variable to its current symbolic value along this branch. `path_conditions` is the list of branch constraints accumulated to reach this configuration; their conjunction is the path condition. `input_translation` maps each input variable to the timestamped symbol allocated for the current step (e.g. `reset_t0`). `new_state_cache` is a scratch map used during a step to hold freshly computed next-state values before they are committed to `state_translation`.

2.2.2 Initialization Procedure

The `init` method creates the single starting path and populates the type maps:

```

pub fn init(&mut self, ctx: &mut Context) {
  for state in &self.ts.states {
    self.state_types.insert(state.symbol, state.symbol.get_type(ctx));
  }
  for input in &self.ts.inputs {
    self.input_types.insert(*input, input.get_type(ctx));
  }

  let mut initial_path = ExecutionPath::new();

  for state in &self.ts.states {
    if let Some(init_expr) = state.init {
      initial_path.state_translation.insert(state.symbol, init_expr);
    } else {
      let name = ctx.get_symbol_name(state.symbol).unwrap_or("state");
      let t0_name = format!("{}",_t0", name);
      let tpe = self.state_types[&state.symbol];
      let t0_sym = match tpe {
        Type::BV(width) => ctx.bv_symbol(&t0_name, width),
        Type::Array(arr_type) => ctx.array_symbol(
          &t0_name, arr_type.index_width, arr_type.data_width),
      };
      initial_path.state_translation.insert(state.symbol, t0_sym);
      self.all_symbols.insert(t0_sym);
    }
  }

  self.constraints = self.ts.constraints.clone();
  self.paths.push(initial_path);
  self.current_step = 0;
}

```

The procedure works in two phases. First, it records the type of every state variable and input by iterating over the transition system and inserting into `state_types` and `input_types`. These maps are consulted later whenever a fresh symbol of the correct width must be allocated.

Second, it builds the single initial `ExecutionPath`. For each state variable the BTOR2 model provides an explicit `init` expression (e.g. `_resetCount ← 0`), that concrete value is used directly. For state variables with no explicit initializer (e.g. `counter`), a fresh unconstrained symbol is created with the suffix `_t0` and registered in `all_symbols`. This symbol represents the unknown initial value and will be treated as a free variable by the SMT solver. Finally, the system constraints are copied from the transition system and the single initial path is pushed onto `paths`.

2.3 Simplification

The goal of simplification is to reduce expressions to forms that cannot be simplified any further. This process relies on a set of basic simplification rules that operate only on the surface structure of an expression. These rules are simple and locally applicable. For example, if both operands of an addition are integers, the expression can be simplified to their sum. Similarly, if the condition of an if expression evaluates to true, the entire expression simplifies to the true branch. However, these rules apply only when their conditions are directly satisfied at the top level of the expression. If an operand is itself a more complex expression (for example, an addition where one operand is symbolic), the rule does not apply. Simplification therefore proceeds by repeatedly applying these basic rules wherever they match. The process continues until no rule can be applied anymore, at which point the expression has reached a fixed point and is considered fully simplified.

To simplify expressions and expression references, we need to start by simplifying the inner most layer. This is implemented with a stack. Each child of an expression gets added to a stack and the last children added is popped and simplified. Table 7 traces this algorithm on the expression *If(true, If(add(0, 1), 4, 5), -1)*: the first column is the step (number of pops), the second is the current result overall (the full expression with all completed simplifications applied so far), the third is the current simplification target (the expression just popped), and the fourth is the stack (top to bottom) after pushing that target's children, if any. The expression simplifies to 4: *-1*, 4, and 5 do not simplify; *add(0, 1)* simplifies to 1; *If(1, 4, 5)* simplifies to 4 (condition is true); *If(true, 4, -1)* simplifies to 4.

Step	Current result overall	Current simplification target	Stack
0	<i>If(true, If(add(0, 1), 4, 5), -1)</i>	<i>If(true, If(add(0, 1), 4, 5), -1) → ?</i>	<i>true, If(add(0, 1), 4, 5), -1</i>
1	<i>If(true, If(add(0, 1), 4, 5), -1)</i>	<i>-1 → -1</i>	<i>true, If(add(0, 1), 4, 5)</i>
2	<i>If(true, If(add(0, 1), 4, 5), -1)</i>	<i>If(add(0, 1), 4, 5) → ?</i>	<i>true, add(0, 1), 4, 5</i>
3	<i>If(true, If(add(0, 1), 4, 5), -1)</i>	<i>5 → 5</i>	<i>true, add(0, 1), 4</i>
4	<i>If(true, If(add(0, 1), 4, 5), -1)</i>	<i>4 → 4</i>	<i>true, add(0, 1)</i>
5	<i>If(true, If(1, 4, 5), -1)</i>	<i>add(0, 1) → 1</i>	<i>true, If(1, 4, 5)</i>
6	<i>If(true, 4, -1)</i>	<i>If(1, 4, 5) → 4</i>	<i>true</i>
7	4	<i>true → true</i>	$\emptyset$

Figure 7: Stack-based simplification trace for *If(true, If(add(0, 1), 4, 5), -1)*. At each step the target is popped and its children (if any) are pushed. *-1*, 4, 5, 0, 1, and *true* do not simplify; *add(0, 1) → 1*, then *If(1, 4, 5) → 4*, then *If(true, 4, -1) → 4*. Result: 4.

2.4 Finding nested if-statements

Using the idea for simplification, the same method can be used to traverse through nested if-statement. Start by implementing a stack which holds all the expressions, a stack which holds all the traversed paths, and a map which holds the position of each expression reference. For each expression, add each child to the mapping, and then explore each children. If any of the children are an ITE expression, we should simplify the path to either the true or false expression and add this to the path stack. Then, with the assumption that the *ite* is true or false, we can use simplification to simplify the expression and add it to the expression stack. Do this for each path as well and eventually we will have a list of paths which represent all feasible combinations of paths and, for each path, the new states.

Consider the expression $add(ite(x, 1, 2), ite(y, 2, 3))$ where x and y are unconstrained symbolic booleans. There are two nested ITE sub-expressions, one inside each argument of add , producing four feasible execution paths. Figure 8 shows the resulting path-exploration tree.

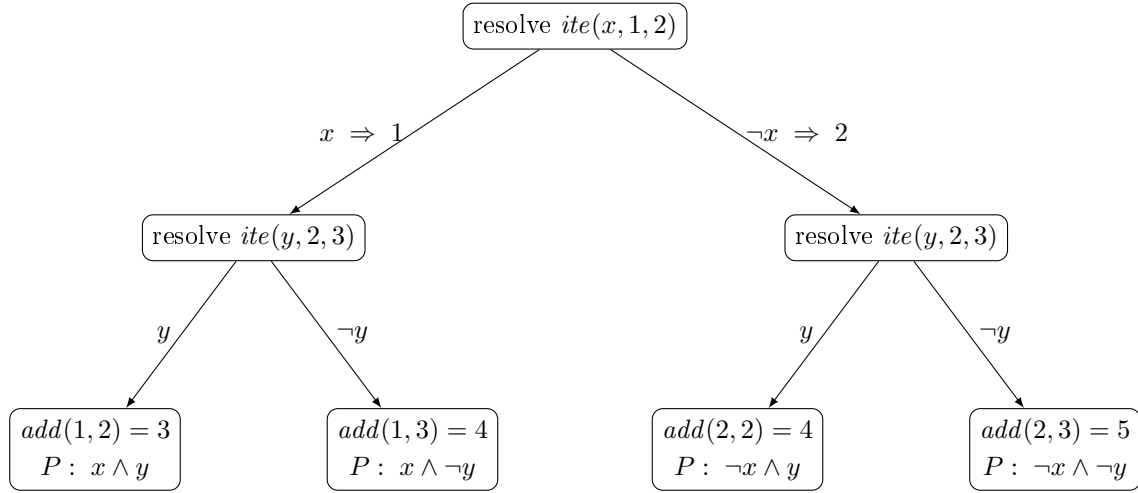


Figure 8: Path-exploration tree for $add(ite(x, 1, 2), ite(y, 2, 3))$. Each internal node shows an ITE being resolved; edges are labelled with the branch condition and its concrete value. Leaves show the fully resolved result and the accumulated path conditions P .

2.5 Pruning Redundant Branches

Not every *ite* expression necessarily results in a branch during symbolic execution. Consider the expression $ite(x > 2, ite(x > 1, 0, 1), 1)$: the condition of the inner *ite* is already entailed by the outer condition, rendering one of its branches unreachable. To handle this precisely, we issue the following queries to the SMT solver:

1. $P \wedge c = SAT \wedge P \wedge \neg c = SAT$: The SMT solver says that both paths are feasible. In this case, we create two branching executions one with $P' = P \wedge c$ and one with $P' = P \wedge \neg c$.
2. $P \wedge c = SAT \wedge P \wedge \neg c = UNSAT$: The SMT solver says that the path such that $\neg c$ is impossible. This means that given the information of P , $\neg c$ is infeasible. As such, we do not need to add $\neg c$ into the path, as the information is already recorded in the path. You can think of this as a node with one branch. It has no other choice so no branching executions are created. Consider $P = (x == 10) \wedge (y == 11)$, $c = (x + y > 12)$. There is no need to add c to the execution path because the value is already included in P .

3. $P \wedge c = UNSAT \wedge P \wedge \neg c = SAT$: The SMT solver says that the path such that c is impossible. Similar to the previous case, we do not need to add c onto the path as the information for deducing $\neg c$ is already included in P .
4. $P \wedge c = UNSAT \wedge P \wedge \neg c = UNSAT$: The result states that regardless of c the execution is infeasible. This must mean that P is infeasible, which means the constraints were infeasible.

2.6 Taking a step

2.7 Path Merging

The current implementation of symbolic execution with just one *ite* expression will double in size with each step. Consider the following state: $x := ite(i > 0, x + 1, x - 1)$. Given the constraint that i is an input and that i is either 1 or 0, there exists two branches in each step. As such within 5 steps, the number of symbolic execution paths will be $2^5 = 32$.

To counter state explosion, we employ path merging. In the previous example, consider the branch that the input is first 0 then 1 versus first 1 then 0. Both symbolic paths result in the same system state. As a result, we can combine the two paths with an $\wedge$ to prevent state explosion.

Figure 9 illustrates the first two steps of this example (x_0 is the initial value of x). Without merging, the four paths at step 2 are tracked separately. With merging, the two paths that reach $x = x_0$ (one via $i_1=0, i_2=1$ and one via $i_1=1, i_2=0$) are combined, reducing the live path count.

Step	i sequence	Value of x	Path condition	Merged?
0	—	x_0	$\top$	—
1	$i_1=1$	$x_0 + 1$	$i_1 > 0$	No
1	$i_1=0$	$x_0 - 1$	$\neg(i_1 > 0)$	No
2	$i_1=1, i_2=1$	$x_0 + 2$	$i_1 > 0 \wedge i_2 > 0$	No
2	$i_1=1, i_2=0$	x_0	$i_1 > 0 \wedge \neg(i_2 > 0)$	} merged
2	$i_1=0, i_2=1$	x_0	$\neg(i_1 > 0) \wedge i_2 > 0$	
2	$i_1=0, i_2=0$	$x_0 - 2$	$\neg(i_1 > 0) \wedge \neg(i_2 > 0)$	No
2 (merged)	$i_1 \neq i_2$	x_0	$(i_1 > 0 \wedge \neg(i_2 > 0)) \vee (\neg(i_1 > 0) \wedge i_2 > 0)$	✓

Figure 9: Path explosion and merging for $x := ite(i > 0, x + 1, x - 1)$ over two steps. The two step-2 paths that reach x_0 are merge candidates; their path conditions are joined with $\vee$.

3 Conclusion

References

- [1] Jonathan Aldrich. Symbolic execution. Lecture notes 14, Course 17-355, Carnegie Mellon University, 2018. Accessed: 2025.
- [2] Kevin Laeuffer. Patronus. GitHub repository, 2024. Accessed: 2025.
- [3] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. BTOR2, BtorMC and Boolector 3.0. In *Computer Aided Verification (CAV)*, volume 10981 of *Lecture Notes in Computer Science*, pages 587–595. Springer, 2018.